European Journal of Operational Research 243 (2015) 61–69

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/ejor

Discrete Optimization

A dynamic programming approach for the aircraft landing problem with aircraft classes

Alexander Lieder [a,*], Dirk Briskorn [b], Raik Stolletz [a]

[a] University of Mannheim, Business School, Chair of Production Management, Germany
[b] University of Wuppertal, Schumpeter School of Business and Economics, Department of Production and Logistics, Germany

ARTICLE INFO

Article history:
Received 24 October 2013
Accepted 17 November 2014
Available online 25 November 2014

Keywords:
Aircraft landing scheduling
Scheduling
Dynamic programming
Exact approach

ABSTRACT

The capacity of a runway system represents a bottleneck at many international airports. The current practice at airports is to land approaching aircraft on a first-come, first-served basis. An active rescheduling of aircraft landing times increases runway capacity or reduces delays. The problem of finding an optimal schedule for aircraft landings is referred to as the "aircraft landing problem". The objective is to minimize the total delay of aircraft landings or the respective cost. The necessary separation time between two operations must be met. Due to the complexity of this scheduling problem, recent research has been focused on developing heuristic solution approaches. This article presents a new algorithm that is able to create optimal landing schedules on multiple independent runways for aircraft with positive target landing times and limited time windows. Our numerical experiments show that problems with up to 100 aircraft can be optimally solved within seconds.

1. Introduction

The number of passenger flights and cargo flights has been increasing over recent years and is expected to continue to increase. The number of aircraft in use, the number of passengers carried, and the air cargo tonnage are expected to double within the next two decades (Boeing, 2013). An important limitation in aviation, however, is the runway systems of airports, which limit the number of take-offs and landings per hour. The runway capacity of major European airports is exceeded in periods of high demand, which leads to delays in take-offs and landings. The cost of delays incurred by air traffic flow management (ATFM) (*i.e.,* during take-off, flight, or landing) for all European airports was estimated to be as high as 1.25 billion euros (1.61 billion dollars) in 2011 (Cook & Tanner, 2011). The total ATFM delay cost in North America was estimated to be as high as 4.6 billion dollars in 2010 (Ball et al., 2010).

The number of possible landings per hour depends on the types of aircraft involved and on the sequence of operations. Depending on its size and shape, each aircraft causes air turbulence ("wake vortices") that affects the following aircraft. Therefore a minimum separation time between two operations is required. Aircraft are usually divided into a small number of aircraft classes. Table 1 shows a matrix of class-dependent minimum separation times. The values in this matrix are based upon the spacing requirements imposed by the US Federal Aviation Administration (FAA, 2012). Different separation matrices can be found in the related literature, but most of these matrices consist of three to five aircraft classes and have a similar structure (Beasley, Sonander, & Havelock, 2001; Harikiopoulo & Neogi, 2011; Psaraftis, 1980; Soomer, 2008).

The aircraft landing problem (ALP) assigns landing times and runways to a given set of aircraft approaching an airport. The planning horizon is very short as the mean time of an aircraft from the time it arrives within the radar range of an airport (the Terminal Maneuvering Area, TMA) to the targeted landing time is approximately 30 minutes (Balakrishnan & Chandran, 2010). As each aircraft has a preferred landing time, the objective is to minimize the total delay costs for all aircraft landings while respecting the separation requirements. The cost function approximates the actual costs such as fuel, maintenance, exhaust emissions, and passengers missing their connecting flights.

By re-arranging the sequence of runway operations instead of using a priority rule, such as FCFS, a significant reduction of total cost can be achieved. For congested runway systems, this optimization leads to either a reduction of the number of aircraft in holding patterns or to an increase of capacity, *i.e.,* more landings per hour that can be performed. This would lead to a considerable increase in revenue.

Extensive reviews of the literature on the ALP are given by Beasley, Krishnamoorthy, Sharaiha, and Abramson (2000) and Bennell, Mesgarpour, and Potts (2013). Table 2 provides an overview of related articles. The columns in the table show the underlying assumptions of the ALP discussed in the respective articles; most of the articles

* Corresponding author. Tel.: +49 6211811461; fax: +49 6211811579.
E-mail address: lieder@uni-mannheim.de (A. Lieder).

http://dx.doi.org/10.1016/j.ejor.2014.11.027
0377-2217/

Table 1
Separation requirements (in seconds).

		Trailing aircraft		
		Small	Large	Heavy
Leading aircraft	Small	82	69	60
	Large	131	69	60
	Heavy	196	157	96

Source: Balakrishnan and Chandran (2010).

discuss the ALP with a single runway ($R = 1$) while others consider multiple (parallel and independent) runways ($R \geq 1$). The most common *objective* is to minimize the total delay costs, but other objectives (such as minimizing the longest delay or minimizing the makespan) are also presented. The delay costs are determined by *cost functions* that are linear or piecewise linear and convex (*i.e.,* the additional cost per period of delay increases). Regarding the target time, we can distinguish two streams of literature; the target times (T_a) are assumed to be zero or are allowed to be positive. Some of the papers allowing positive target times allow *early landings* to occur, that is, landings between the target time and an earliest landing time (E_a), which are also associated with costs. Most articles assume limited time windows for landings, *i.e.,* there is a *latest landing time* (L_a) for each aircraft that must not be exceeded by its actual landing time. The last column shows which solution approaches are discussed in the respective articles.

To date, no efficient methods have been proposed in the reviewed literature for the multi-runway ALP that are capable of solving large problem instances. The most common solution approaches are (1) mixed-integer programming (MIP) formulations, which are solved with a standard solver; (2) branch-and-bound (B&B) algorithms; (3) dynamic programming (DP) approaches; and (4) heuristic solution approaches.

MIP formulations: the first mixed-integer formulation for the ALP on a single runway was published by Abela, Abramson, Krishnamoorthy, De Silva, and Mills (1993). The extension to multiple runways by Beasley et al. (2000) is the most cited MIP formulation of the ALP to date. Pinol and Beasley (2006) further generalize this formulation to runway-dependent time windows and separation times. Briskorn and Stolletz (2014) proposed a modification of the MIP of Beasley et al. (2000) that explicitly considers aircraft classes.

B&B algorithms: Abela et al. (1993) present a B&B approach for the single-runway ALP. Ernst, Krishnamoorthy, and Storer (1999) develop a B&B solution procedure for the ALP that outperforms standard solvers using the MIP formulation by Beasley et al. (2000) but, nevertheless, results in excessive computation times for all instances, except for small problem instances.

Dynamic programming approaches: Dear (1976) presents a dynamic programming formulation for the single runway problem with a constrained-position-shifting (CPS) assumption, *i.e.,* each aircraft can be shifted only by a limited number of positions from the sequence of arrivals at the runway. CPS approaches are also presented by Psaraftis (1980), Dear and Sherif (1991), and, more recently, by Balakrishnan and Chandran (2010). Psaraftis (1980) presents a DP approach for the single-runway ALP that makes use of the class-dependent separation times to reduce the problem complexity. Bianco, Dell'Olmo, and Giordani (1999) present a DP approach for a single-machine scheduling problem with sequence-dependent setup times that is equivalent to the single-runway case of the ALP without consideration of aircraft classes. Briskorn and Stolletz (2014) describe a DP approach for the ALP with multiple runways and under consideration of limited time windows. However, they do not provide numerical results for their approach.

Heuristic solution approaches: Abela et al. (1993) propose a genetic algorithm (GA) as a heuristic solution approach. Bianco et al. (1999) propose two heuristic approaches (cheapest addition and cheapest insertion) for their DP approach. Fahle, Feldmann, Gtz, Grothklags, and Monien (2003) compare different exact and heuristic solution approaches for the ALP on a single runway: MIP, integer programming (IP), constraint programming (CP), hill climbing (HC), and simulated annealing (SA). Pinol and Beasley (2006) develop two population-based heuristic approaches (scatter search and a bionomic algorithm) for the ALP. Soomer (2008) and Soomer and Franx (2008) introduce fairness aspects to the ALP by providing airlines the opportunity to define their own cost functions. The numerical study is performed using a local search heuristic.

Many articles assume a fixed number of aircraft classes, but only a few actually use this property in their solution approaches (Bojanowski, Harikiopoulo, & Neogi, 2011; Briskorn & Stolletz, 2014; Harikiopoulo & Neogi, 2011; Psaraftis, 1980). The algorithms presented in the remaining articles assume aircraft-dependent cost functions and separation requirements. However, most of the problems discussed feature class-dependent cost functions and separation requirements.

This article contributes to the current state of research in the scheduling of airport runway operations by providing a new optimization algorithm for the ALP with general assumptions (multiple runways, positive target times, and limited time windows). Existing approaches are either heuristic or rely on more restrictive assumptions concerning the number of runways, landing times, or time windows, see Table 2. Our method is based on the DP approach by Briskorn and Stolletz (2014). However, this paper focuses on computational complexity only. While polynomiality of the developed approach is proven, it is left open how to efficiently implement the approach and what actual computation times can be achieved. Certainly, computation times can be expected to be huge for a straightforward implementation since the run time complexity is reported to be $O(n^{17})$ (n being the number of aircraft) for two runways and two aircraft classes. We purposefully modify the problem setting in order to allow more efficient solution methods without losing practice-orientation. Also, we develop a new dominance criterion for state space reduction by which we can reduce computation times significantly. We obtain a DP approach yielding optimum schedules significantly faster than a standard MIP solver (CPLEX).

Our approach does not consider stochastic events, *e.g.,* changes in target landing times of approaching aircraft or weather conditions. We assume short planning horizon of approximately 45–60 minutes, in which it is possible to calculate the target landing times with a high precision, and to have a reliable weather forecast.

In Section 2, we provide a formal problem definition as well as a mixed-integer programming formulation. In Section 3, the new optimization algorithm is described in detail. The numerical study in Section 4 compares the results of the algorithm to optimal results of a MIP solver and provides a sensitivity analysis. Section 5 briefly describes how the algorithm could be used in practice by embedding it in a rolling horizon approach. Section 6 summarizes the major insights and outlines future research.

2. Problem definition

2.1. Problem description

We consider a set, $A = \{1, \ldots, |A|\}$, of aircraft partitioned into disjoint subsets, $A_1, \ldots, A_W$, of W classes and a set of R identical and independent runways. Each aircraft, $a \in A$, belongs to exactly one class of aircraft in $\{1, \ldots, W\}$ denoted by $w(a)$ and has a target landing time, T_a, and a latest possible landing time, L_a. As in Briskorn and Stolletz (2014), we assume throughout the paper that there is no pair (a, a') of aircraft with $w(a) = w(a')$, $T_a < T_{a'}$, and $L_a > L_{a'}$.

For each class w, a non-decreasing and convex cost function, $c_w(d) : \mathbb{R}_+ \to \mathbb{R}$, reflects the additional cost depending on the deviation, d, of the actual landing time from the target time of an aircraft of

Table 2
Overview of related articles.

Source	R	Objective	Cost function	Time windows	Solution approach(es)
Dear (1976)	1	min Σ costs	Linear	$E_a = T_a = 0;\ L_a = \infty$	Dynamic program (with CPS)
Psaraftis (1980)	1	min Σ costs/min makespan	Linear	$E_a = T_a = 0;\ L_a = \infty$	Polynomial dynamic program (with "FCFS within classes")
Dear and Sherif (1991)	1	min Σ delay/min makespan	Linear	$E_a = T_a \geq 0;\ L_a = \infty$	Dynamic program (with CPS)
Abela et al. (1993)	1	min Σ costs	Linear	$E_a \leq T_a \leq L_a$	MIP, exact B&B, heuristic (GA)
Ernst et al. (1999)	≥1	min Σ costs	Linear	$E_a \leq T_a \leq L_a$	MIP, exact B&B, problem search space (PSS) heuristic
Bianco et al. (1999)	1	min Σ delay	Linear	$E_a = T_a \geq 0;\ L_a = \infty$	(exponential) dynamic program, heuristics (addition, insertion)
Beasley et al. (2000)	≥1	min Σ costs	Piecewise linear, convex	$E_a \leq T_a \leq L_a$	MIP, IP
Fahle et al. (2003)	1	min Σ costs	Linear	$E_a \leq T_a \leq L_a$	Overview: MIP, IP, CP, heuristics (HC, SA)
Pinol and Beasley (2006)	≥1	min Σ costs	Linear	$E_a \leq T_a \leq L_a$	MIP, scatter search, bionomic algorithm
Soomer (2008)	1	minmax delay costs	Piecewise linear, convex	$E_a = T_a \geq 0;\ L_a = \infty$	MIP, local search heuristic
Soomer and Franx (2008)	1	minmax delay costs	Piecewise linear, convex	$E_a \leq T_a \leq L_a$	MIP, local search heuristic
Balakrishnan and Chandran (2010)	1	min makespan	n/a	Arbitrary	Dynamic program (with CPS)
Briskorn and Stolletz (2014)	≥1	min Σ costs	Piecewise linear, convex	$E_a \leq T_a \leq L_a$	Polynomial dynamic programs, MIP (with "FCFS within classes")

class w. Finally, for each pair (w, w') of classes, a minimum separation time, $s_{w,w'}$, is given.

A solution to the ALP is a schedule $S \subset A \times \{1, \ldots, R\} \times \mathbb{R}$ with exactly one $(a, r, t) \in S$ for each $a \in A$. A triple $(a, r, t) \in S$ represents a being scheduled on r at time t. For $(a, r, t) \in S$ and $(a', r, t') \in S$ with $t' \geq t$ such that no $(a'', r, t'') \in S$ with $t < t'' < t'$ exists we say that a' immediately follows a.

A solution S is called feasible if

- for each $(a, r, t) \in S$ we have $T_a \leq t \leq L_a$, that is, each aircraft lands in its landing window,
- for each pair (a, a') of aircraft such that a' immediately follows a, we have $(a, r, t) \in S$ and $(a', r, t') \in S$ with $t' - t \geq s_{w(a),w(a')}$, that is, the minimum separation time is satisfied.

The problem, then, is to find a feasible solution S which minimizes

$$\sum_{(a,r,t)\in S} c_{w(a)}(t - T_a) \tag{1}$$

among all feasible solutions. We refer to this problem as the *ALP* in the following sections.

2.2. MIP model

In this section, we present a MIP formulation of the ALP as defined in Section 2.1. The sets, parameters, and variables of the model are shown in Table 3. We use binary variables, $\gamma_{a,a'}$, to indicate if aircraft a' immediately follows aircraft a. Additional binary variables, f_{ar} and l_{ar}, indicate the first and last aircraft on each runway. Variable C_a represents the assigned landing time for each aircraft.

The MIP with successive separation requirements can be stated as follows.

$$\text{Minimize } F = \sum_{a\in A} c_{w(a)}(C_a - T_a) \tag{2}$$

subject to the constraints

$$T_a \leq C_a \leq L_a \quad \forall a \in A \tag{3}$$

$$C_a + s_{w(a)w(a')} \leq C_{a'} + M(1 - \gamma_{aa'}) \quad \forall a, a' \in A; a \neq a' \tag{4}$$

$$C_a \leq C_{a'} \quad \forall a, a' \in A; T_a < T_{a'}; w(a) = w(a') \tag{5}$$

$$\sum_{a'\in A} \gamma_{a'a} + \sum_{r=1}^{R} f_{ar} = 1 \quad \forall a \in A \tag{6}$$

Table 3
Sets, parameters, and variables of the MIP model.

Sets	
A	Set of aircraft a to be scheduled
Parameters	
W	Number of different aircraft classes w
R	Number of (identical) runways r
$w(a)$	Class of aircraft $a \in A$: $w(a) \in \{1, \ldots, W\}$
T_a	Target landing time of aircraft $a \in A$
L_a	Latest possible landing time of aircraft $a \in A$
$s_{w,w'}$	Minimum separation time between classes w and w'
M	A sufficiently large number
Decision Variables	
C_a	Assigned landing time for aircraft $a \in A$
$\gamma_{aa'}$	$= \begin{cases} 1 & \text{if } a' \text{ immediately follows } a \\ 0 & \text{otherwise} \end{cases}$ $\quad \forall (a, a') \in A \times A,\ a \neq a'$
f_{ar}	$= \begin{cases} 1 & \text{if } a \text{ lands } \textit{first} \text{ on runway } r \\ 0 & \text{otherwise} \end{cases}$ $\quad \forall a \in A,\ r = 1, \ldots, R$
l_{ar}	$= \begin{cases} 1 & \text{if } a \text{ lands } \textit{last} \text{ on runway } r \\ 0 & \text{otherwise} \end{cases}$ $\quad \forall a \in A,\ r = 1, \ldots, R$

$$\sum_{a'\in A} \gamma_{aa'} + \sum_{r=1}^{R} l_{ar} = 1 \quad \forall a \in A \tag{7}$$

$$\sum_{a\in A} f_{ar} \leq 1 \quad \forall r = 1, \ldots, R \tag{8}$$

$$\sum_{a\in A} l_{ar} \leq 1 \quad \forall r = 1, \ldots, R \tag{9}$$

$$C_a \geq 0 \quad \forall a \in A \tag{10}$$

$$\gamma_{aa'}, f_{ar}, l_{ar} \in \{0; 1\} \quad \forall a, a' \in A \quad \forall r = 1, \ldots, R \tag{11}$$

The objective function (2) sums up the total delay cost of all aircraft landings incurred by the delay of the respective aircraft's scheduled landing time, C_a, from its target time, T_a. Constraint (3) ensures that each landing is scheduled within the respective time window, $[T_a, L_a]$. The separation requirement for all pairs of subsequent aircraft that land on the same runway is ensured by constraint (4): if $\gamma_{aa'} = 1$, *i.e.*, a lands immediately before a' on the same runway, the respective landing times, C_a and $C_{a'}$, must be separated by at least $s_{w(a)w(a')}$. Otherwise, if $\gamma_{aa'} = 0$, the equation is valid for a large enough M, *e.g.*, $M \geq L_a + S_{w(a)w(a')} - T_{a'}\ \forall a, a' \in A$.

A key property of aircraft classes is that an FCFS sequence within each class can be assumed (Briskorn & Stolletz, 2014). In (5), this

property is implemented by forcing aircraft of a particular class to land in increasing order of target times. Constraints (6) and (7) ensure that each aircraft, a, has exactly one predecessor (unless it is the first aircraft landing on its runway) and exactly one successor (unless it is the last aircraft). Each runway has at most one aircraft landing first and one aircraft landing last, as stated in (8) and (9).

3. Dynamic programming approach

3.1. States and transitions

We propose a dynamic programming algorithm based on the framework proposed by Briskorn and Stolletz (2014). Briskorn and Stolletz (2014) developed a DP approach to prove that the ALP can be solved polynomially in $|A|$ but exponentially in W and R. Their approach was not implemented, as they argue that the size of the state space is too large for a straightforward implementation.

We define each state of the dynamic program as a tuple $(k_1, \ldots, k_W, rop)$, where

- k_w, $w = 1, \ldots, W$, is the number of aircraft of class w that have been scheduled, and
- rop is a *runway occupation profile* (ROP). It is defined as a vector $((O_1, w_1), \ldots, (O_R, w_R))$ that contains the time O_r and aircraft class w_r of the latest landing on each runway r.

A runway with no operations scheduled is denoted in a ROP as $(-1, -1)$. Note that the state tells us which aircraft have already been scheduled to land due to the FCFS assumption within each class, and the earliest possible time of the next landing for each class on each runway.

The initial state of the program is $(0^W, (-1, -1)^R)$, *i.e.,* no landings have been scheduled yet. A feasible state transition

$$(k_1, \ldots, k_W, (O_1, w_1), \ldots, (O_R, w_R)) \Rightarrow (k'_1, \ldots, k'_W, (O'_1, w'_1), \ldots, (O'_R, w'_R)) \quad (12)$$

corresponds to the scheduling of the $(k_w + 1)$th aircraft in class w, namely a, on runway r, that is, we have

- $k'_{w'} = k_{w'}$ for each $w' \neq w$,
- $k'_w = k_w + 1 \leq |A_w|$,
- $(O'_{r'}, w'_{r'}) = (O_{r'}, w_{r'})$ for each $r' \neq r$, and
- $(O'_r, w'_r) = (\max\{T_a, O_r + s_{w_r,w}\}, w)$ with $O'_r \leq L_a$.

This transition is associated with a cost, $c_{w(a)}(O'_r - T_a)$. The cost of a state s can be defined via a **Bellman recursion** as

$$Z(s) = \min_{s' \in \Pi(s,a,t)} (Z(s') + c_{w(a)}(t - T_a)) \quad (13)$$

where $\Pi(s, a, t)$ is the set of states for which a feasible transition to s by landing a at time t exists, and $Z(0^W, (-1, -1)^R) = 0$.

We consider the set Π of terminal states with $s = (|A_1|, \ldots, |A_W|, rop)$ for each $s \in \Pi$. The ALP then can be solved by finding a state

$$s^* = \arg\min_{s \in \Pi} \{Z(s)\}. \quad (14)$$

The actual schedule can be derived by tracking the sequence of transitions that transform the initial state into s^*, inducing $Z(s^*)$.

This DP approach corresponds to the approach by Briskorn and Stolletz (2014). Both the states and the transitions are defined in a similar fashion. However, because we do not consider *early landings* incurring additional costs, the number of transitions and the state space of the DP are significantly smaller. In Briskorn and Stolletz (2014), for each transition there is a multitude of possible landing times for the next scheduled landing. Without early landings, it is optimal to schedule the next landing as early as possible. Different papers on the aircraft landing problem state that the benefit of early landings is limited. Thus they either assume that an aircraft's target landing time is also the earliest possible (Bianco et al., 1999; Venkatakrishnan, Barnett, & Odoni, 1993), or they limit the possible earliness to a small value, e.g., 60 seconds, (Kupfer, 2009; Lee & Balakrishnan, 2008). Our approach allows early landings, if they are not associated with additional costs.

3.2. State-space reduction using a dominance criterion

This section develops a reduction of the state space that is actually searched by employing a dominance criterion and by removing symmetry. We traverse the state space in order by considering states with the smallest number of aircraft being landed yet first. After all states with q aircraft scheduled have been evaluated, we then proceed to those states having $q + 1$ aircraft scheduled to land. However, before proceeding to states with $q + 1$ aircraft, we sort the R entries of the ROP for each state with q aircraft according to the non-decreasing class of the last aircraft that landed and use the last landing time as a tie-breaker. Note that this does not change the basis for scheduling further aircraft because we assume that the runways are identical and independent. This sorting step will lead to states with symmetric ROPs to be identified in the check for dominated states described as follows.

Using the ROP of a state $(k_1, \ldots, k_W, rop)$ and the separation requirements, we define $p_{wr} = O_r + s_{w_r,w}$ as the earliest possible landing time for the $(k_w + 1)$th aircraft of class w on runway r for all runways $r = 1, \ldots, R$ and all aircraft classes $w = 1, \ldots, W$.

We say that ROP rop dominates ROP rop' ($rop \succ rop'$) if each runway, r, is available for the next aircraft of each class, w, earlier or at the same time, *i.e.,* if $p_{wr} \leq p'_{wr}$ holds for each $w = 1, \ldots, W$ and $r = 1, \ldots, R$. Then we say that a state, s, dominates another state, s', ($s \succ s'$) if

- at least the same number of aircraft k_w of each class w have already been scheduled ($k_w(s) \geq k_w(s')$ for each $w \in W$), and
- the cost of s does not exceed the cost of s' ($Z(s) \leq Z(s')$), and
- the ROP of state s dominates the ROP of state s' ($rop(s) \succ rop(s')$).

A dominated state s' can be removed from further consideration, that is, we do not consider any transition that starts in s'.

Note that we should (at least implicitly) check for dominance between each of the pairs of states that is reached. We can reduce the computational burden of these dominance checks, as we carefully traverse the state space by evaluating the states in a non-decreasing order of aircraft that are scheduled. It is then easy to observe that only the dominance needs to be checked between states having the same number of aircraft scheduled for each class.

4. Numerical study

In Section 4.1, we demonstrate the efficiency of the new algorithm by comparing its computation time with that of a standard MIP solver using the formulation presented in Section 2.2. We use two standard data sets from the scientific literature (Bianco et al., 1999) and 10 randomly generated, realistic data sets.

We solve each problem instance using four different cost functions:

- linear increase of delay cost (with rate 1 monetary unit/second)
- no delay cost during first minute, then linear increase (with rate 1 monetary unit/second)
- no delay cost during first 5 minutes, then linear increase (with rate 1 monetary unit/second)
- cost increase doubling every 5 minutes (with rate 1 monetary unit/second during the first 5 minutes)

Table 4
Standard problem instances from Bianco et al. (1999).

			New algorithm		MIP w/ CPLEX			
Problem instance	Cost function	R	Objective value (Z)	CPU time (seconds)	Objective value (Z)	CPU time (seconds)	Gap to lower bound (percent)	Gap to opt. objective (percent)
1	Linear	1	3721	<1	3721	>3600	76	0
		2	219	<1	219	2	0	0
		3	0	<1	0	1	0	0
1	1 minute	1	3342	<1	3342	>3600	87	0
	free, then	2	0	<1	0	<1	0	0
	linear	3	0	<1	0	2	0	0
1	5 minutes	1	313	<1	313	107	0	0
	free, then	2	0	<1	0	2	0	0
	linear	3	0	<1	0	<1	0	0
1	Doubling	1	4034	<1	4034	>3600	56	0
	every 5	2	219	<1	219	2	0	0
	minutes	3	0	<1	0	2	0	0
2	Linear	1	20 391	<1	23 246	>3600	95	14
		2	660	<1	660	23	0	0
		3	63	<1	63	3	0	0
2	1 minute	1	19 432	<1	20 442	>3600	100	5
	free, then	2	21	<1	21	5	0	0
	linear	3	0	<1	0	3	0	0
2	5 minutes	1	10 473	<1	11 097	>3600	100	6
	free, then	2	0	<1	0	21	0	0
	linear	3	0	<1	0	4	0	0
2	Doubling	1	42 555	<1	54 134	>3600	100	27
	every 5	2	660	<1	660	25	0	0
	minutes	3	63	<1	63	4	0	0

The second and third cost functions can be used to allow early landings (without additional costs). We assume a limited time window of 30 minutes for each aircraft ($L_a = T_a + 30$ minutes) in all problem instances in this study. We performed additional tests with smaller time windows ($L_a = T_a + 15$) minutes and open time windows ($L_a = \infty$), but we could not observe significant changes in the computation times, neither for the MIP, nor for our DP approach.

Next, in Section 4.2 we analyze the sensitivity of the algorithm's performance to problem size (40, 50, 60, 70, 80, 90, and 100 aircraft) and to the average inter-arrival times of 30, 35, 40, 50, and 60 seconds. We solve 10 randomly generated data sets for different combinations of the aforementioned parameters on $R = 2,\ 3,$ and 4 runways. In Section 4.3, we show the impact of the state-space reduction presented in Section 3.2 on the computation times.

For all randomly generated problem instances, we use the separation matrix in Table 1. All calculations are performed on an Intel Core i5 computer (2.5 gigahertz, 8 gigabytes RAM). The MIP formulations are solved with CPLEX 12.2; our new algorithm is implemented with Java JDK 1.6. The computation time limit is set to 60 minutes for each problem instance.

4.1. Performance analysis

Bianco et al. (1999) provide two realistic sets of aircraft with target times and separation requirements. These sets are also used in numerical studies, *e.g.,* by Ernst et al. (1999) and by Briskorn and Stolletz (2014). Set 1 consists of $|A| = 30$ aircraft divided into $W = 4$ classes, and set 2 consists of $|A| = 44$ aircraft divided into $W = 2$ classes. By scheduling each of these sets on $R = 1,\ 2,$ and 3 runways, we obtain a total of six problem instances.

Table 4 compares the computation times of our algorithm and of the MIP formulation from Section 2.2. The "gap to lower bound" shows the difference between the best feasible solution found by CPLEX after 1 hour and the respective lower bound. The "gap to opt. objective" shows the difference between the solution of our algorithm and the best feasible solution found by CPLEX. Our new algorithm optimally solves all instances in less than a second, regardless of the cost function. For $R = 1$, CPLEX cannot find proven optimal solutions within 1 hour, with one exception.

For the second comparison, we generate 10 data sets using the following parameters. Each data set consists of $|A| = 50$ aircraft divided into three classes, with the separation matrix shown in Table 1. By analyzing the inbound traffic of nine major American airports, Willemain, Fan, and Ma (2004) show that a Poisson arrival process (*i.e.,* exponentially distributed interarrival times) is a realistic approximation of the inbound traffic of an airport. We use one set of exponentially distributed interarrival times in 10 different randomized orders, with the first target time, T_a, of 0. Therefore, the last target time, $\max_a T_a$, is the same for all 10 resulting profiles. Each target time is randomly assigned to an aircraft class out of a set of exactly 20 percent small aircraft, 40 percent large aircraft, and 40 percent heavy aircraft, which is a realistic configuration (Balakrishnan & Chandran, 2010). We choose an average interarrival time of 40 seconds. The resulting distributions of target times and aircraft classes for all 10 profiles are shown in Fig. 1.

Table 5 shows the optimal solutions to our randomly generated problem instances for all four cost functions. Each profile is scheduled on $R = 2, 3,$ and 4 runways. Due to the limited time windows, none of the problem instances has a feasible solution for $R = 1$.

For the linear cost function, our new algorithm outperforms the MIP solver in all instances. All instances are solved in less than 1 minute, while CPLEX exceeds the 1 hour time limit in most cases with $R = 2$ and 3. In particular, for $R = 2$ the gap to the optimal solution is quite large. When $R = 4$, the runway utilization is unrealistically low and the two have a comparable performance.

For the cost functions "doubling every 5 minutes" and "1 minute free, then linear", the computation times are not significantly different from the linear cost function. For the cost function "5 minutes free, then linear", our approach obtains fast and optimal results for $R = 2$ while the MIP hits the time limit in all cases and returns non-optimal solutions. For cases with low runway utilization, $R = 3$ and $R = 4$, however, the optimal objective value is 0, that is, all aircraft can land with less than 5 minutes delay. The MIP can solve these cases in seconds, while our approach takes up to 5 minutes for $R = 4$. For the

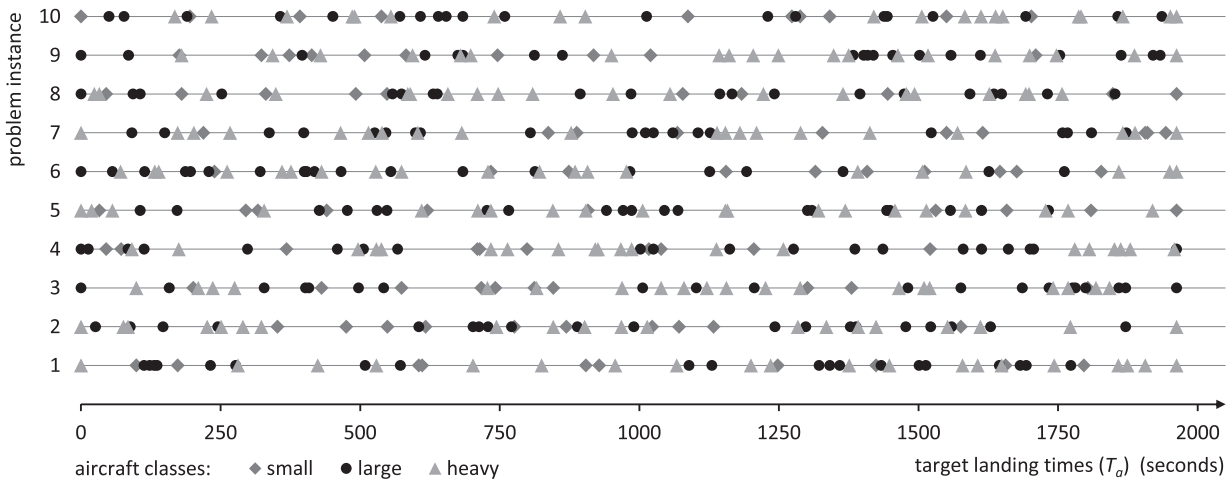

Fig. 1. Distribution of target landing times and aircraft classes.

most important cases with high runway utilization, that is, the cases where sequencing has a real impact, the DP finds optimal solutions within seconds regardless of the shape of the cost function.

4.2. Sensitivity analysis

We tested how the **number of aircraft** affects the computation times for problem instances with $|A| = 40, 50, 60, 70, 80, 90$, and 100 aircraft, using a linear cost function. The other parameters remain unchanged (40 seconds average interarrival times, 30 minutes time windows). Table 6 shows the average and maximum computation times for 10 problem instances of each size. For $|A| = 60$ aircraft or less, the computation took less than 1 minute in all of the test cases. The average computation time increases with the number of aircraft, but even for $|A| = 100$ aircraft, the 1 hour time limit was never reached.

To observe how the **load on the runway system** affected the computation times, we used problem instances with $i = \{30, 35, 40, 50, 60\}$ seconds for problems with $|A| = 50$ aircraft and a 30 minute time window. Table 7 shows the computation times for 10 problem instances for each configuration. For short interarrival times ($i = 30$) and $R = 4$ runways, the time limit was exceeded in one problem instance. If this particular instance is excluded, the average computation time for $i = 30$ and $R = 4$ is only 19 seconds. All of the problem instances with an average interarrival time of 40 seconds or longer were solved in under 1 minute.

4.3. Impact of state space reduction

The following tests show that the algorithm presented in Section 3.1 performs well only with the state-space reduction presented in Section 3.2. We generated a set of small problem instances, with $|A| = 10, 15$, and 20 aircraft, with the same parameters as our problem instances from Section 4.1. We solved these instances both with and without state space reduction. Table 8 shows the average computation times (in seconds) and the average number of states created for 10 problem instances in each setting. The table entries ">3600" or "n/a" indicate that the algorithm did not return a solution after 1 hour.

Without state-space reduction, the dynamic solution approach is not solvable. The number of states created is up to 400 times larger than with state-space reduction. The state space contains a vast number of symmetric and sub-optimal states that make even small problem instances intractable.

5. Implementation via a rolling horizon approach

As stated in the introductory section, we assume to have precise data on the upcoming aircraft landings 45–60 minutes in advance, as soon as the aircraft reach the *terminal manoeuvering area* (TMA) of the airport. When the respective aircraft reach their *final approach path*, about 20 minutes before landing, their trajectory and thus their position relative to other aircraft may no longer be changed. Thus, for each runway, the sequence of aircraft entering the final approach path is the same as the sequence of landings. The time in between (that is, 25–40 minutes) can be used to optimize the order and the runway assignment of the approaching aircraft.

A number of authors (*e.g.*, Venkatakrishnan et al., 1993, Furini, Persiani, and Toth (2012)) propose to adapt their solution approaches to a dynamic scenario by embedding it in a rolling horizon framework: The landing schedule for the aircraft in the TMA, which have not reached the final approach path yet, is optimized; then, iteratively, after a certain amount of time passed, the schedule of the aircraft closest to the final approach path is fixed and a new schedule is calculated, taking into account the aircraft that entered the TMA in the meantime.

In peak times, major European airports like Frankfurt/Main or London-Heathrow have about 90 aircraft movements per hour, or about 40–60 aircraft in a time window of 25–40 minutes, respectively. With the ability to optimally schedule this number of aircraft in less than a minute, it is possible to use our solution approach in this iterative fashion, as well. To ensure the separation requirements between the fixed schedule and the schedule obtained in the subsequent iteration, the state, that represents the time and aircraft class of the last scheduled landing on each runway, has to be used as the initial state of the next iteration of the optimization algorithm.

6. Conclusions and further research

This article presents a dynamic programming algorithm that is capable of efficiently solving the ALP with different aircraft classes on multiple runways with positive target landing times and limited time windows. Based on a DP formulation by Briskorn and Stolletz (2014), we develop a dominance criterion that eliminates states from the state space while maintaining optimality. The numerical study reveals that the algorithm quickly and optimally solves large problem instances and outperforms the standard MIP solver, CPLEX. The study also shows that the new dominance criterion significantly improves the performance of the dynamic programming approach.

Table 5
Comparison of computation times for 10 problem instances and 4 cost functions.

Cost function		Linear						1 minute free, then linear						5 minutes free, then linear						Doubling every 5 minutes					
		DP		MIP w/ CPLEX				DP		MIP w/ CPLEX				DP		MIP w/ CPLEX				DP		MIP w/ CPLEX			
Inst.	R	Z	CPU (seconds)	Z	CPU (seconds)	Gap (LB) (percent)	Gap (opt.) (percent)	Z	CPU (seconds)	Z	CPU (seconds)	Gap (LB) (percent)	Gap (opt.) (percent)	Z	CPU (seconds)	Z	CPU (seconds)	Gap (LB) (percent)	Gap (opt.) (percent)	Z	CPU (seconds)	Z	CPU (seconds)	Gap (LB) (percent)	Gap (opt.) (percent)
1	2	5410	3	7267	>3600	100	26	3562	4	3647	>3600	98	2	174	9	300	>3600	100	42	5932	4	6181	>3600	95	4
	3	1027	3	1037	>3600	72	1	178	5	178	81	0	0	0	33	0	9	0	0	1027	3	1027	>3600	64	0
	4	233	5	233	28	0	0	0	16	0	7	0	0	0	82	0	5	0	0	233	5	233	15	0	0
2	2	7277	7	10 628	>3600	100	32	6164	6	7248	>3600	99	15	194	31	1867	>3600	100	90	7979	9	8228	>3600	93	3
	3	871	2	876	>3600	72	1	23	3	23	16	0	0	0	22	0	8	0	0	871	2	871	>3600	71	0
	4	109	4	109	13	0	0	0	19	0	10	0	0	0	66	0	7	0	0	109	3	109	8	0	0
3	2	5220	2	11 445	>3600	100	54	2973	2	3251	>3600	97	9	117	10	194	>3600	100	40	5448	2	5900	>3600	89	8
	3	966	3	1008	>3600	63	4	249	5	249	157	0	0	0	27	0	8	0	0	966	3	966	>3600	62	0
	4	328	10	328	44	0	0	22	34	22	12	0	0	0	280	0	5	0	0	328	10	328	38	0	0
4	2	6002	2	7242	>3600	100	17	3910	3	4439	>3600	98	12	24	9	49	>3600	100	51	6234	2	7639	>3600	97	18
	3	970	2	970	>3600	70	0	234	3	234	32	0	0	0	30	0	9	0	0	970	2	970	>3600	52	0
	4	222	6	222	19	0	0	0	18	0	9	0	0	0	87	0	5	0	0	222	5	222	10	0	0
5	2	5256	3	7960	>3600	100	34	3663	2	4201	>3600	98	13	0	9	769	>3600	100	100	5521	3	5698	>3600	95	3
	3	454	1	454	2652	0	0	45	2	45	10	0	0	0	15	0	7	0	0	454	1	454	905	0	0
	4	42	3	42	9	0	0	0	11	0	8	0	0	0	32	0	7	0	0	42	3	42	5	0	0
6	2	8646	3	12 115	>3600	100	29	7098	3	7832	>3600	99	9	357	20	1565	>3600	100	77	9686	4	11 989	>3600	96	19
	3	928	2	928	>3600	71	0	160	3	160	59	0	0	0	47	0	8	0	0	928	2	928	>3600	61	0
	4	245	2	245	16	0	0	0	9	0	7	0	0	0	42	0	5	0	0	245	2	245	13	0	0
7	2	6096	2	8358	>3600	100	27	3753	2	3971	>3600	98	5	0	10	70	>3600	100	100	6254	3	7628	>3600	92	18
	3	765	2	765	>3600	57	0	51	2	51	10	0	0	0	12	0	8	0	0	765	2	771	>3600	40	1
	4	147	4	147	11	0	0	0	10	0	8	0	0	0	33	0	6	0	0	147	4	147	7	0	0
8	2	5859	3	8478	>3600	100	31	4124	2	5665	>3600	98	27	0	13	100	>3600	100	100	6136	3	7862	>3600	92	22
	3	532	2	532	2875	0	0	50	2	50	23	0	0	0	17	0	8	0	0	532	2	532	1489	0	0
	4	109	3	109	10	0	0	0	14	0	9	0	0	0	40	0	6	0	0	109	3	109	4	0	0
9	2	6491	3	8914	>3600	100	27	4080	2	5005	>3600	99	18	53	19	717	>3600	100	93	6777	3	7547	>3600	97	10
	3	884	3	889	>3600	54	1	44	6	44	25	0	0	0	48	0	6	0	0	884	3	886	>3600	57	0
	4	219	10	219	24	0	0	0	36	0	10	0	0	0	291	0	6	0	0	219	10	219	9	0	0
10	2	5063	2	6115	>3600	100	17	2983	1	3214	>3600	97	7	0	7	40	>3600	100	100	5287	2	5436	>3600	88	3
	3	590	1	590	>3600	38	0	20	3	20	14	0	0	0	17	0	7	0	0	590	2	590	824	0	0
	4	139	3	139	10	0	0	0	19	0	8	0	0	0	55	0	5	0	0	139	4	139	5	0	0

Table 6
Computation times for the different problem sizes (in seconds).

Problem	$R = 2$		$R = 3$		$R = 4$	
size $\|A\|$	Average	Maximum	Average	Maximum	Average	Maximum
40	1	1	1	2	2	4
50	4	10	3	4	7	15
60	14	43	12	37	15	37
70	28	80	12	28	22	45
80	85	335	25	73	51	252
90	140	611	29	58	54	91
100	392	1420	41	178	92	524

Table 7
Computation times for the different interarrival times (in seconds).

Average	$R = 2$		$R = 3$		$R = 4$	
inter-arrival time	Average	Maximum	Average	Maximum	Average	Maximum
30	29	66	117	517	753	>3600
35	23	65	7	23	8	14
40	4	10	3	4	7	15
50	1	1	2	2	4	6
60	1	1	1	1	2	3

Table 8
Impact of the state-space reduction on computation time and state space.

		Computation time (seconds)		Number of states created	
$\|A\|$	$\|R\|$	Without state space reduction	With state space reduction	Without state space reduction	With state space reduction
10	2	0.1	0.0	365 060	938
10	3	2.3	0.1	384 820	1643
10	4	32.8	0.1	436 789	2665
15	2	2.4	0.1	1646 941	4124
15	3	276.9	0.1	1952 312	5620
15	4	>3600	0.2	n/a	8882
20	2	23.1	0.1	911 586	9648
20	3	>3600	0.1	n/a	10 690
20	4	>3600	0.2	n/a	15 534

In future research, we will extend the solution approach to require less restrictive assumptions than are discussed in the scientific literature regarding the ALP: Some authors allow early landings ($C_a < T_a$) with penalty costs, assume discontinuous landing time windows (Artiouchine, Baptiste, & Dürr, 2008), assume heterogeneous and interdependent runways, or assume both, take-offs and landings. For the latter, the ground holding problem (Rossi & Smriglio, 2001) has to be taken into consideration. In addition, our approach may serve as a framework for heuristic solution methods such as beam search, to make even larger problems tractable.

Acknowledgments

This research was partially supported by the Erich-Becker-Stiftung, a foundation of the Fraport AG, and by the Julius Paul Stiegler Memorial Foundation.

References

Abela, J., Abramson, D., Krishnamoorthy, M., De Silva, A., & Mills, G. (1993). Computing optimal schedules for landing aircraft. In *Proceedings of the 12th national conference of the Australian Society for Operations Research* (pp. 71–90), Adelaide.

Artiouchine, K., Baptiste, P., & Dürr, C. (2008). Runway sequencing with holding patterns. *European Journal of Operational Research*, *189*(3), 1254–1266.

Balakrishnan, H., & Chandran, B. (2010). Algorithms for scheduling runway operations under constrained position shifting. *Operations Research*, *58*(6), 1650–1665.

Ball, M., Barnhart, C., Dresner, M., Hansen, M., Neels, K., Odoni, A., et al. (2010). Total delay impact study: A comprehensive assessment of the costs and benefits of flight delay in the United States: Technical report. National Center of Excellence for Aviation Operations Research (NEXTOR).

Beasley, J., Krishnamoorthy, M., Sharaiha, Y., & Abramson, D. (2000). Scheduling aircraft landings-the static case. *Transportation Science*, *34*(2), 180–197.

Beasley, J., Sonander, J., & Havelock, P. (2001). Scheduling aircraft landings at London Heathrow using a population heuristic. *Journal of the Operational Research Society*, *52*(5), 483–493.

Bennell, J., Mesgarpour, M., & Potts, C. (2013). Airport runway scheduling. *Annals of Operations Research*, *204*(1), 249–270.

Bianco, L., Dell'Olmo, P., & Giordani, S. (1999). Minimizing total completion time subject to release dates and sequence-dependent processing times. *Annals of Operations Research*, *86*, 393–415.

Boeing (2013). *Current market outlook*. http://www.boeing.com/commercial/cmo/.

Bojanowski, L., Harikiopoulo, D., & Neogi, N. (2011). Multi-runway aircraft sequencing at congested airports. In *American control conference (ACC)* (pp. 2752–2758).

Briskorn, D., & Stolletz, R. (2014). Aircraft landing problems with aircraft classes. *Journal of Scheduling*, *17*(1), 31–45.

Cook, A., & Tanner, G. (2011). European airline delay cost reference values: Technical report. EUROCONTROL Performance Review Unit.

Dear, R. (1976). The dynamic scheduling of aircraft in the near terminal area: Technical report. Cambridge, MA: Flight Transportation Laboratory, Massachusetts Institute of Technology.

Dear, R., & Sherif, Y. (1991). An algorithm for computer assisted sequencing and scheduling of terminal area operations. *Transportation Research Part A: General*, *25*(2–3), 129–139.

Ernst, A., Krishnamoorthy, M., & Storer, R. (1999). Heuristic and exact algorithms for scheduling aircraft landings. *Networks*, *34*(3), 229–241.

FAA (2012). *American federal aviation administration safety alert for operators (safo) #12007*. http://www.faa.gov/other_visit/aviation_industry/airline_operators/airline_safety/safo/.

Fahle, T., Feldmann, R., Gtz, S., Grothklags, S., & Monien, B. (2003). The aircraft sequencing problem. In R. Klein, H.-W. Six, & L. Wegner (Eds.), *Lecture notes in computer science: Vol. 2598. Computer science in perspective* (pp. 152–166). Berlin/Heidelberg: Springer.

Furini, F., Persiani, C., & Toth, P. (2012). Aircraft sequencing problems via a rolling horizon algorithm. In *Combinatorial optimization* (pp. 273–284).

Harikiopoulo, D., & Neogi, N. (2011). Polynomial-time feasibility condition for multi-class aircraft sequencing on a single-runway airport. *IEEE Transactions on Intelligent Transportation Systems*, *12*(1), 2–14.

Kupfer, M. (2009). Scheduling aircraft landings to closely spaced parallel runways. In *Eighth USA/Europe air traffic management research and development seminar, Napa, CA.*

Lee, H., & Balakrishnan, H. (2008). A study of tradeoffs in scheduling terminal-area operations. *Proceedings of the IEEE*, *96*(12), 2081–2095.
Pinol, H., & Beasley, J. (2006). Scatter search and bionomic algorithms for the aircraft landing problem. *European Journal of Operational Research*, *171*(2), 439–462.
Psaraftis, H. N. (1980). A dynamic programming approach for sequencing groups of identical jobs. *Operations Research*, *28*(6), 1347–1359.
Rossi, F., & Smriglio, S. (2001). A set packing model for the ground holding problem in congested networks. *European Journal of Operational Research*, *131*(2), 400–416.
Soomer, M. (2008). *Runway operations scheduling using airline preferences* (Ph.D. dissertation). Amsterdam: VU University.
Soomer, M., & Franx, G. (2008). Scheduling aircraft landings using airlines' preferences. *European Journal of Operational Research*, *190*(1), 277–291.
Venkatakrishnan, C., Barnett, A., & Odoni, A. (1993). Landings at Logan airport: Describing and increasing airport capacity. *Transportation Science*, *27*(3), 211–227.
Willemain, T., Fan, H., & Ma, H. (2004). Statistical analysis of intervals between projected airport arrivals: Technical report. Pennsylvania State University, Department of Decision Sciences and Engineering Systems.